# Chatrail — Project Context Packet

**Generated:** 2026-05-05 **Source repo:** `/Users/blainefisher/Documents/ChatQuest` (package name `chatquest`, product name `Chatrail`) **Deployed at:** https://chattrail.netlify.app

---

---

## How to Use This Document

This is a portable project context packet. Upload it to any AI tool — ChatGPT, Claude, Gemini, Codex, Antigravity, Cursor, or another assistant — and the AI will have enough context to help with product, business, technical, marketing, or roadmap work without re-asking the basics.

**Suggested kickoff prompt** (also in §22):

> *I have uploaded a project context packet for this software project. Read it carefully and treat it as the source of truth for our discussion. Help me think through the project using that context. If something is missing, ask targeted questions. Do not assume facts that are not in the packet. When giving recommendations, separate product advice, business advice, technical advice, and risks.*

The packet was generated by inspecting the repository directly (README, ROADMAP, CHANGELOG, SECURITY, package.json, .env.example, all 13 SQL migrations, marketing pages, app-route structure, and source comments). Anything inferred is labeled as such. Anything unknown is labeled "Unknown" or surfaced in §23 Open Questions.

**No secrets are included.** Environment variable names appear where useful but never their values.

---

## 1. Project Identity

| FIELD | VALUE |
|---|---|
| **Project name (product)** | **Chatrail** |
| **Project name (repo / package)** | `chatquest` (legacy — being rebranded to Chatrail) |
| **Repo path** | `/Users/blainefisher/Documents/ChatQuest` |
| **GitHub** | `Bfisher911/ChatQuest` (per SECURITY.md and Netlify config) |
| **Current URL** | https://chattrail.netlify.app (Netlify deploy) |
| **Intended URL (planned)** | `chatquest.app` (mentioned in ROADMAP as a target custom domain — Unknown if registered) |

**Tagline (from landing page / README):** *Chatbot-native LMS for serious learning.*

**Elevator pitch:** Chatrail is a learning-management system where every "lesson" is a chatbot conversation — instructors compose visual learning paths from AI tutor nodes, content readings, PDFs, milestones, and certificates; learners chat their way through; instructors grade transcripts against rubrics with AI assistance; verifiable PDF certificates auto-issue on completion.

**Longer product description:** Chatrail replaces video-and-quiz-style courses with chatbot-native learning. A creator (instructor, trainer, subject-matter expert, or organization admin) builds a "Chatrail" — an ordered graph of nodes. Most nodes are AI-tutor conversations: each has its own system prompt, persona, learner instructions, knowledge base (RAG over uploaded PDFs / TXT / MD / CSV), rubric, and token budget. Other node types include static content readings, PDFs, slides, external links, milestones with prereqs, and certificates. Learners enroll, work through the path, and submit each chatbot node for grading. The instructor (or AI-assisted grader) scores transcripts against per-criterion rubrics and returns feedback. Verifiable PDF certificates issue automatically when a learner satisfies the milestone-cert linkage. The whole platform is multi-tenant: organizations buy seat-based plans through Stripe, instructors bring learners under their seat allocation, and learner-paid programs are a supported alternative.

**Visual identity:** A distinctive **brutalist black-and-white** design system — Space Grotesk + VT323 + Press Start 2P fonts, ink/paper/soft/muted palette, hairline borders, "cassette"-style cards with binary indices like `00000010`. The visual language is unusually confident for an LMS and is part of the product positioning.

**Current stage:** - **Phase 1 MVP shipped** (2026-04-28) on Netlify - **Phases A–I shipped** (2026-04-29 in CHANGELOG): production safety hardening, branded transactional emails, visual path builder, certificates, Stripe billing scaffold, cost control, analytics, engineering hygiene, UX polish, performance, marketing/SEO, plus - **Additional Phase 2-stage features shipped this**

**session** (deployed): publish/draft/archive workflow, permanent Chatrail deletion, gradebook editable per-criterion rubric, gradebook latest-attempt correctness fix, gradebook filter+search+sort, programs list filter+search, dashboard cassette real data, /learn rich progress, per-node grading data on overview, claim-invite-for-current-account, duplicate rubric, live cost estimate in bot editor, org-admin token usage panel, attempts indicator + locked CTA, stop in-flight chat, `use_program_kb` toggle, `ai_grading_enabled` toggle, **AI Chatrail Generator (one-prompt → ready-to-edit Chatrail draft)**

Inferred: the product is in **late MVP / pilot-ready** state. Core flows work end-to-end against live Supabase + LLM providers. Phase 3 Stripe wiring is implemented but real subscription monetization has not yet been validated against paying customers.

**Intended audience category:** - Primary: **commercial multi-tenant SaaS** (educational + corporate L&D) - Also viable as: institutional license (university / school district), individual-instructor, or learner-pays-per-program - Marketing pages explicitly target "for-education" and "for-corporate" segments (see `app/(marketing)/for-education/` and `for-corporate/`)

---

---

## 2. Core Problem

**Problem statement.** Traditional online learning is built around video lectures and multiple-choice quizzes. Both treat the learner as a passive recipient. Cognitive science (and any honest teacher) knows the deepest learning happens through **conversation**: explaining ideas in your own words, getting Socratic pushback, defending positions, debugging mistakes, and being asked penetrating follow-up questions. Until LLMs, simulating that one-on-one conversational instruction at scale was impossible. With LLMs it is — but existing LMS platforms (Canvas, Blackboard, Moodle, Teachable, Thinkific, Kajabi) bolt chatbots on as a side feature. They aren't built around them.

**Who has this problem.** - University instructors who know discussion drives learning but can't give every student one-on-one Socratic time - Corporate L&D teams running technical, sales, or compliance training who need real practice, not click-through compliance theatre - Subject-matter experts (consultants, coaches, healthcare educators, legal trainers) who want to package their expertise as repeatable, scalable instruction - Instructional designers tired of building courses out of slides and form-quizzes - Students/learners who get more out of conversation than passive video, but rarely have access to it

**Why it matters.** Active learning beats passive learning by every educational metric (retention, transfer, motivation). Chatbots can simulate active learning at infinite scale and zero marginal cost. The bottleneck is no longer the medium; it's the **authoring experience** — designing pedagogically-sound chatbot conversations, sequencing them, grading transcripts, and proving completion. That's the gap.

**What happens if not solved.** Educators stay stuck choosing between (a) high-touch low-scale (in-person discussion, expensive consultants), (b) low-touch high-scale (videos + quizzes, low retention), or (c) the rapidly expanding wild-west of "ChatGPT homework" without any instructional framing, grading, or certification. Institutions either lose ground to AI-native upstarts or end up paying enterprise vendors for shoehorned chatbot features.

**How users currently solve it.** Mostly badly. Combinations of: - Ad-hoc ChatGPT in the browser with hand-written system prompts that leak to learners - Custom-built tutor sites (rare, expensive, brittle) - LLM features bolted onto Canvas / Moodle / Blackboard (limited authoring, no grading workflow, weak rubrics) - Single-vendor AI-tutor products that don't compose into multi-step learning paths - Plain LLM chat with no rubric, no transcript grading, no completion proof, no multi-tenant org management

---

---

## 3. Target Users and Customer Segments

### Primary users (people in the app every day)

- **Instructors / Creators / Trainers / SMEs** — author Chatrails, configure bot system prompts, upload KB documents, grade transcripts, manage rosters
- **Learners / Students** — enroll, chat through nodes, submit, view grades + feedback, earn certificates

### Secondary users

- **Teaching Assistants (TAs)** — co-instruct, grade, but typically don't author
- **Co-instructors** — full edit rights on a single program

### Buyer / decision-maker personas

- **Organization admins** — buy plans, allocate seats, manage members, view usage + cost dashboards
- **Department / institutional decision-makers** — university deans, corporate L&D leads, training directors
- **Self-pay individual instructors** — buying an Instructor Basic / Pro / Premium plan for their own teaching

### Internal / platform roles

- **Super admin** — platform operators (orgs, users, subscriptions, abuse review, manual overrides)

- **Public visitor** — landing pages, pricing, features, for-education / for-corporate, certificate verification

### Customer types

- **Multi-tenant organizations** — the canonical buyer (universities, corporate training departments, schools, non-profits, training companies)
- **Individual instructors** — solo creators on Stripe-backed instructor plans
- **Learner-paid program enrollees** — for programs where the program is sold direct to learners (one-time, per-semester, or annual fee)
- **Free-tier users** — capped at 1 active program / 3 chat nodes / 5 learner seats (per the seeded `free` plan)

### Roles found in the codebase (`app.user_role` enum)

`super_admin | org_admin | instructor | ta | learner` — confirmed in `supabase/migrations/00010000000002_identity.sql`.

---

---

## 4. Product Vision

**Long-term vision.** Become the default platform for AI-tutor-driven instruction — the LMS that's chatbot-native rather than chatbot-bolted-on. Where any instructor, coach, expert, or training department can take their pedagogy and turn it into scalable conversational learning, with verifiable completion. The "Notion for AI tutoring" or the "Canvas for chatbot-native courses."

**What it should eventually become.** A platform with: - A polished visual builder where designing a Chatrail feels like designing a flowchart - A library / marketplace of starting templates ("debate coach", "medical history simulator", "constitutional law moot court", "junior-engineer code review") shareable across the community - LMS interoperability via LTI 1.3 so it slots into Canvas / Blackboard / Moodle without ripping them out - Native SCORM / xAPI / certificate integrations so corporate buyers don't have to convince their compliance team - Rich analytics on learning outcomes (mastery curves, common-misconception detection from transcripts, teacher-facing insights from real conversations) - A serious bring-your-own-LLM-key story for cost-sensitive enterprise buyers - Multi-language support so the same pedagogy ships globally

**What makes it useful (today's value prop).** It removes the friction between "I have a teaching idea worth scaling" and "my learners are practicing it conversationally with AI." Authoring, grading, rostering, billing, and certification are all in one platform built specifically around the chatbot-as-lesson model.

**What makes it different from a generic AI wrapper.** - **Pedagogical structure.** Every bot has a rubric, a grading workflow, an attempts policy, and a place in a sequenced learning path. It's not a chat box. - **Multi-tenant SaaS architecture.** Real orgs, seat licensing, RLS, role-aware UI, audit logs, plan gating. - **Transcript-as-artifact grading.** Conversations become gradable submissions. AI suggests rubric scores; instructors override. - **Visual path builder.** Real React Flow graph with conditional unlocks, branching, milestones, and cert nodes. - **Verifiable certificates.** Per-org template, signature, logo, unique verification code, public verification URL. - **Provider-agnostic.** Anthropic / OpenAI / Gemini behind one adapter. Per-bot model selection. Future BYO-key path. - **A confident, opinionated visual identity.** The brutalist B&W design is a deliberate "we are not your enterprise LMS" signal. - **Cost transparency.** Live per-attempt cost estimate in the bot editor; org-admin sees MTD cost on the dashboard; usage logs ledger every LLM call. - **One-prompt Chatrail generator** (just shipped) — describe a course in plain English, get a 3–7 node draft with ready-written system prompts.

**Ideal user experience.** - A trainer types a paragraph describing what they want learners to walk away knowing - A draft Chatrail materializes with prewritten bots, rubrics, and a flow - They tweak each bot inline with a live PREVIEW CHAT panel - They publish, invite learners, and watch progress on a real dashboard - Learners feel like they're talking to a knowledgeable, slightly-strict tutor, not a generic chatbot - Grading takes minutes per submission with the AI summary + suggested rubric scores doing the first pass - Certificates issue automatically and verifiers can confirm them with a public URL

---

---

## 5. Business Model

**Plan structure** (live in `plans` table; seeded by `00010000000008_billing.sql`):

| PLAN CODE | NAME | SCOPE | MONTHLY USD | INSTRUCTOR SEATS | LEARNER SEATS | MONTHLY TOKENS | NOTES |
|---|---|---|---|---|---|---|---|
| `free` | Free | Instructor | $0 | 1 | 5 | 50,000 | 1 active program, 3 chat nodes, no certs |
| `instr_basic` | Instructor Basic | Instructor | $19 | 1 | 30 | 200,000 | 3 programs, 15 chat nodes, certs |
| `instr_pro` | Instructor Pro | Instructor | $49 | 1 | 100 | 600,000 | 10 programs, 50 chat nodes |
| `instr_premium` | Instructor Premium | Instructor | $99 | 1 | 300 | 1,800,000 | 50 programs, 200 chat nodes |
| `org_starter` | Organization Starter | Org | $299 | 3 | 150 | 900,000 | 20 programs |
| `org_dept` | Department Plan | Org | $799 | 10 | 500 | 3,000,000 | 100 programs |
| `org_school` | School Plan | Org | $1,999 | 30 | 2,000 | 9,000,000 | 500 programs, **SSO** |
| `org_enterprise` | Enterprise | Org | Contact sales | Custom | Custom | Custom | SSO, custom seats |

**Pricing logic (multi-axis):** - **Seat-based** — instructor seats + learner seats per plan - **Usage-based gate** — monthly token budget caps spending per org - **Feature-flag** — `certificates`, `sso`, `max_active_programs`, `max_chat_nodes` are flag-gated via the `plans.features` jsonb column - **Learner-paid programs (alternative)** — a program can have `learner_pays = true` so the instructor-side seat allocation is bypassed and learners pay (one-time, semester, or annual) via Stripe Checkout

**Revenue paths:** 1. **Self-serve instructor plans** — credit card, low-friction, $19–$99/mo 2. **Self-serve org plans** — bigger deals, $299–$1,999/mo, still card 3. **Enterprise sales** — `org_enterprise` is `contact_sales: true`, expects bespoke contracts 4. **Learner-paid programs** — instructor sets a price, Chatrail takes a platform fee (Inferred — fee % is **Unknown**) 5. **Future:** marketplace transactions on shared templates / Chatrails (long-term, see §4)

**Free tier mechanics:** new self-serve signups land on `free`, capped at 1 program / 3 chat nodes / 5 learners / 50K tokens / no certs. Designed to give a real taste of the product without enabling actual classroom use.

**Trial / freemium / pilot:** - Free tier exists and is full-featured below the caps - Inferred: pilot deals likely happen as comp'd `org_dept` or higher (Unknown — no formal trial flag in the `plans` schema) - ROADMAP mentions "BLOCKED ON YOU" Stripe-account setup; full Checkout is implemented but live products + price IDs need to be created on Stripe

**Buyer approval flow:** - Self-serve: instant via Stripe Checkout - Enterprise: contact-sales link → manual onboarding → super admin sets `plan_code` (Inferred from schema)

**Upsells / premium:** - Mid-tier plan upgrades happen via Stripe Customer Portal ( `/api/stripe/portal` ) - Token-budget overage forces a soft 80% / hard 100% block on chat — explicit cost-management upsell pressure - SSO is gated to `org_school` and above — common B2B upsell lever

**Status of monetization wiring:** Stripe Checkout, webhook ( `/api/stripe/webhook` ), and Customer Portal all exist as routes. Plan-gating helpers ( `canCreateProgram` , `canSeatLearner` , `canIssueCertificate` , `allowedModelsForPlan` in `lib/billing/gate.ts` ) are wired into the create / invite paths. **What's not done yet:** real Stripe products + prices need to be set up by the founder (BLOCKED ON YOU items in ROADMAP), and DNS/email-domain pre-work is incomplete.

---

---

## 6. Market and Competitive Positioning

**Market category:** AI-native LMS / AI tutoring platform / chatbot-native learning platform. Sits in the overlap of three larger markets: LMS ($25B+, growing), corporate L&D / training ($300B+), and AI-tutoring tools (newer, fragmented).

**Direct competitors (chatbot-native or AI-tutor-led):** - **Khanmigo** (Khan Academy's AI tutor) — K-12-focused, single-tenant institutional, not a platform - **Quizlet AI / Duolingo Max** — single-vendor, single-domain - **PowerNotes / SchoolAI / Magic School / Brisk Teaching** — teacher-side AI tools, not full LMS workflow - **AceUp / Stepful / Storia AI** — vertical AI-tutor products - **Heuristics / Mentava** — AI-tutor-as-product - **Custom GPTs / OpenAI Custom GPTs** — strong DIY but no rubric, no LMS, no certificates, no multi-tenancy - **Mentor.AI / various YC startups** in the AI-tutor space (rapid churn — list is volatile)

**Indirect competitors:** - **Canvas / Blackboard / Moodle** (incumbent LMS) — adding chatbot features, but not chatbot-native - **Teachable / Thinkific / Kajabi / Podia** (creator-economy LMS) — strong on courses, weak on conversation + AI - **Articulate / Cornerstone / Docebo** (corporate LMS) — bolting on AI; not the design starting point - **Notion / Coda + Custom GPTs** — DIY hack that some teachers cobble together

**Why someone would choose Chatrail:** - It's **chatbot-native** — built around the conversation, not bolted on - The **visual path builder + rubric grading + certificates** stack is unusually complete for the AI-tutor segment - **Multi-tenant SaaS from day one** — most AI-tutor competitors are single-tenant or don't have an org / seat / billing story - **Provider-agnostic LLM** — instructors aren't locked to one model vendor; can swap per node; cost-sensitive cohorts can use Haiku, depth-needs use Sonnet - **Cost transparency** — both at the instructor level (live cost estimate in bot editor) and the org-admin level (MTD usage panel). Not a black box. - **One-prompt generator** lowers the activation barrier from "blank page panic" to "edit a draft" - **Brutalist confident UI** — communicates "we know what we're doing" in a category dominated by enterprise-bland or kid-cartoon design

**What makes it easier / cheaper / more focused / faster:** - **Easier than DIY ChatGPT + spreadsheet:** real rubrics, transcripts, gradebook, certificates, rosters, billing - **Cheaper than enterprise LMS for AI tutoring:** $19–$1,999 plans vs. five- to six-figure enterprise contracts - **More focused than Canvas/Blackboard:** doesn't try to be everything; doesn't carry their UI tax - **Faster to author:** AI generator + inline preview + sensible defaults - **More transparent than black-box AI tutors:** instructors see the prompts, the costs, and own the IP

**Possible acquisition / partnership targets (long-term, speculative):** - Acquirers: Anthropic, OpenAI, Khan Academy, Coursera, 2U/edX, Instructure (Canvas), Cengage, Pearson, Cornerstone OnDemand - Partners: K-12 districts, university teaching-and-learning centers, corporate L&D consultancies, instructional-design agencies, AI-cert / digital-credential issuers

---

---

# 7. Product Scope

## What the app currently does (as built)

- Sign up / sign in / forgot password / accept invite (with the just-added "claim invite for current account" path)
- Role-aware dashboards: super_admin, org_admin, instructor, learner — TA dashboard shares instructor view (Inferred)
- Create + edit + duplicate + delete Chatrails (with title-typed confirmation for delete)
- **Visual path builder** (React Flow) with 7 node types: bot, content, pdf, slides, link, milestone, cert

- Bot node editor: system prompt, learner instructions, model picker (Claude / OpenAI / Gemini), temperature, token budget, max tokens, attempts, points, rubric assignment, KB toggle, AI-grading toggle, **inline PREVIEW CHAT panel**, live cost estimate
- **Knowledge base**: PDF / TXT / MD / CSV upload, chunking, embedding (OpenAI 1536-dim or Gemini 768→1536 padded), pgvector retrieval via `match_embeddings` Postgres RPC
- **SSE chat streaming** with provider auto-routing by model name; per-message token logging; per-attempt + monthly token caps; **stop in-flight response** button
- **Submission + grading**: AI summary + suggested rubric scores; editable per-criterion grader panel with rationales; instructor comment; status (graded / needs_revision / excused / pending_review)
- **Rubrics**: create, edit criteria + max points, duplicate full rubric (with criteria + levels), delete with attached-bot guard
- **Roster**: invite by email (with copyable link panel after send), CSV bulk import with preview + per-row validation, manage learners + instructors, suspend/reactivate
- **Gradebook**: full matrix with filter pills (ALL / PENDING / GRADED / REVISION / MISSING), name search, sort, latest-attempt-wins correctness, AI grading suggest, CSV export
- **Certificates**: cert template per org (signer name, custom body text, org logo upload), `@react-pdf/renderer` PDFs, public verification at `/verify-cert/[code]`, auto-award on milestone + min-grade rule
- **Org admin**: members management, billing, settings, cert-template editor, MTD token usage panel
- **Super admin**: orgs / users / subscriptions / usage ledger
- **Learner experience**: rich `/learn` hub (progress, due dates, last activity, cert count), journey page with progress engine + locked rail awareness, attempts indicator, friendly attempts-exhausted screen, due-date surfacing, budget-exhausted banner
- **Stripe**: Checkout (org + instructor + learner-paid program), webhook (subscription sync, plan-code mirror, past_due), Customer Portal
- **Cost control + analytics**: per-program analytics page (Recharts), token-usage `getMonthlyTokenUsage`, rate limits via Upstash Redis with in-memory fallback
- **Engineering hygiene**: vitest (19 tests passing), GitHub Actions CI, Sentry wiring, structured JSON logger, `/api/health`
- **Marketing**: landing page, features, pricing (driven from `plans` table via PostgREST), for-education, for-corporate, /privacy, /terms, /aup, OG image, robots.txt, sitemap
- **AI Chatrail Generator** (just shipped): one-prompt → multi-node Chatrail draft with prewritten system prompts, learner instructions, sensible model + budget + attempts defaults, linear edges. Lands in DRAFT for editing.

## What the app does not do yet

- Real Stripe **products + price IDs** (BLOCKED ON YOU — code is wired, account setup is pending)
- **DOCX uploads** for KB (only PDF / TXT / MD / CSV today)
- **Email confirmation** (currently disabled in `supabase/config.toml` for dev — toggle for prod)
- **Email sending** without `RESEND_API_KEY` falls back to console
- **LTI 1.3 / SCORM / xAPI** integration (not in MVP)
- **SSO** is gated to `org_school+` plans but the SAML / OIDC IdP integration code is not yet present (Inferred — no LTI / SAML files in the route tree)
- **Mobile app** — responsive web only
- **Multi-language UI** (English only)
- **BYO LLM key per org** (planned in J.1)
- **Marketplace / template library** (long-term)
- **Audit-log viewer UI** (logs are written; the `/admin/audit` page is roadmapped J.3)

## MVP definition (what's working today vs the spec)

**Phase 1 MVP** (workflow 1) is fully working: instructor signs up, builds bot, uploads KB, invites learner, learner chats and submits, instructor grades, learner sees grade. Confirmed in `README.md` "Manual smoke test" section.

**Phases A–I** plus this session's work are deployed on top of MVP, so the live app is materially beyond Phase 1.

## What's planned for later (from ROADMAP.md)

- Phase J: legal scaffolds (already partially done — /terms /privacy /aup pages exist), data export + account deletion (routes exist), audit log viewer UI, ops runbooks
- Real Stripe + DNS + custom domain
- BYO LLM key
- Org-level KB collections (currently program-level)
- Per-node attempt-count UI enforcement
- Performance: ISR is in place; bundle audit + further indexes ongoing

## Out of scope for now

- Native iOS / Android apps
- Live video / synchronous learning

- Peer review / discussion forums
- Marketplace transactions
- White-label / on-prem deployments
- Non-English UI

## Important assumptions

- Multi-tenant by org from day one — no plan to retro-fit later
- Postgres + Supabase as the long-term DB / auth / storage / vector layer
- Next.js App Router on Netlify Lambdas — RAG had to be moved off direct-Postgres because Lambda outbound :5432 is unreliable
- Brutalist visual identity is core branding, not a draft — Inferred from how completely the design system is implemented

---

## 8. Key Features

| FEATURE | WHAT IT DOES | WHO USES IT | WHY IT MATTERS | STATUS |
|---|---|---|---|---|
| **Visual path builder** | React Flow graph with drag-drop palette, custom node renderers, edge drawing + condition editor | Creators | Authoring is the core job — this is where the product earns its category | Complete |
| **Bot node editor** | Title, learner instructions, system prompt, model picker, temp, budget, max tokens, attempts, points, rubric, KB toggle, AI-grading toggle | Creators | Where pedagogy gets encoded | Complete |
| **Inline PREVIEW CHAT** | Ephemeral chat using the saved bot config — no real conversation row, no submission, no grade pollution | Creators | Test-drive before invite — closes a major authoring loop gap | Complete |
| **Live cost estimate** | Per-attempt + per-learner cost for current model + budget + attempts | Creators | Choosing gpt-4o vs claude-haiku used to be cost-blind | Complete |
| **AI Chatrail generator** | One-prompt → 3–7 node Chatrail draft with system prompts and instructions written | Creators | Removes the blank-page activation barrier; collapses authoring time from hours to minutes | Complete (just shipped) |

| FEATURE | WHAT IT DOES | WHO USES IT | WHY IT MATTERS | STATUS |
|---|---|---|---|---|
| **Knowledge base + RAG** | Upload PDF/TXT/MD/CSV; auto-chunk, embed (OpenAI or Gemini); pgvector retrieval via Postgres RPC; cited inline | Creators (upload), Bots (retrieve) | Source-grounded bots; provable answers | Complete |
| **SSE streaming chat** | Token-by-token streaming, multi-provider, per-attempt + monthly budget caps, abort/stop button | Learners | The actual learning surface | Complete |
| **Submission + grading** | Learner submits → AI summary + suggested rubric scores → instructor grader panel with editable per-criterion scores + rationales → grade visible to learner with rubric breakdown | Learners + Creators | Transcript-as-artifact grading is what makes this an LMS, not a chat app | Complete |
| **Editable per-criterion rubric** | Each criterion has its own score input + rationale textarea; SUM TO TOTAL convenience button | Creators | Detailed, rubric-aligned feedback rather than total-only | Complete |
| **Rubrics library** | Create, edit, duplicate (criteria | Creators | | Complete |

| FEATURE | WHAT IT DOES | WHO USES IT | WHY IT MATTERS | STATUS |
|---|---|---|---|---|
| | + levels), delete with attached-bot guard | | Reusable across programs | |
| **Roster + invites** | Email invite (with copyable link panel after send), CSV preview + commit, suspend/ reactivate, claim-for-current-account flow | Creators + invitees | Solves the "did the email arrive" failure mode and the "I already have an account" friction | Complete |
| **Bulk CSV import** | Paste CSV → preview with per-row validation (valid / duplicate / invalid_email / missing_email) → confirm | Creators | Onboarding cohorts at scale | Complete |
| **Gradebook** | Matrix view, filter pills, name search, sort, latest-attempt-wins, AI suggest, CSV export | Creators | Triage-first workflow | Complete |
| **Certificates** | Per-org template (signer, body, logo), auto-award on milestone+min-grade, PDF + verification URL | Creators + Learners + Verifiers | Completion proof has real-world value | Complete |
| **Multi-provider LLM adapter** | Claude / OpenAI / Gemini behind one interface, model picker per bot | Creators | No vendor lock-in; cost flexibility | Complete |

| FEATURE | WHAT IT DOES | WHO USES IT | WHY IT MATTERS | STATUS |
|---|---|---|---|---|
| **Stripe billing** | Org plan + instructor plan + learner-paid program Checkout; webhook; Customer Portal; plan-feature gates wired into create/invite | Buyers + Org admins | Monetization | Complete (code), pending real Stripe products |
| **Token budget caps + cost ledger** | Soft warn at 80%, hard 402 at 100%, per-row `est_cost_usd` in `usage_logs` | Org admins | Prevents budget surprises | Complete |
| **Org-admin token usage + MTD cost panel** | Live progress bar + dollar number on org dashboard | Org admins | Surfaces the biggest billing anxiety | Complete |
| **Per-program analytics** | Tokens-per-day line chart + avg score per node bar (Recharts) | Creators | Outcome visibility | Complete |
| **Lifecycle controls** | Draft / published / archived; permanent delete with title-typed confirmation; learner access guards on draft/ archived | Creators | Production-quality state machine | Complete |
| **Notifications hub** | In-app notification rows (invite_received, grade_returned, | Everyone | Closing the loop on async events | Complete (Inferred — schema migration `00010000000040_phase_t_notifications.sql` exists; UI route `app/(app)/notifications/` ) |

| FEATURE | WHAT IT DOES | WHO USES IT | WHY IT MATTERS | STATUS |
|---|---|---|---|---|
| | etc.) + bell counter | | | |
| **Account export + delete** | Self-serve data export + account deletion routes | Everyone | GDPR-ready user-rights compliance | Complete ( `/api/account/export` , `/api/account/delete` ) |
| **Health check** | `/api/health` probes Supabase + LLM + Stripe | Ops | Monitoring | Complete |
| **Sentry + structured logs** | Error monitoring + JSON log lines | Ops | Production debuggability | Complete (no-op without `SENTRY_DSN` ) |
| **GitHub Actions CI** | typecheck → lint → vitest → next build on every push | Engineering | Don't ship broken | Complete |
| **Brutalist design system** | Cassette / Chip / Btn / Icon / Eyebrow / Frame primitives + `cq-*` CSS layer | Everywhere | Brand differentiation | Complete |

Inferred / unknown features (mentioned in ROADMAP / CHANGELOG but quality not verified): - **Audit log viewer UI** at `/admin/audit` — Planned (J.3); rows are written but the viewer is not built per ROADMAP - **Bring-your-own-LLM-key per org** — Planned (J.1) - **DOCX KB uploads** — Planned (Phase 2 follow-on) - **LTI / SSO IdP** — SSO flag exists at the plan tier but actual SAML/OIDC integration is **Unknown / Not Built**

---

---

## 9. User Roles and Permissions

The role enum lives at `app.user_role` in `00010000000002_identity.sql` :

```
super_admin | org_admin | instructor | ta | learner
```

Plus a platform-level boolean `users.is_super_admin`. Role assignment is per-org via `organization_members.role`.

### Super admin

**What they can do:** see and edit everything across all orgs; access `/admin/orgs`, `/admin/users`, `/admin/subscriptions`, `/admin/usage`; manually override plans, seat counts, role assignments; bypass cross-org guards. **Toggle behavior:** super admins still log in like any other user; the role switcher in the header dropdown lets them act as a member of any org they're enrolled in. Inferred: they bypass plan gating (`session.user.isSuperAdmin` is checked in cross-org checks).

### Organization admin

**What they can do:** - Manage org members + invites at `/org/members` - Edit org settings + cert template at `/org/settings`, `/org/cert-template` - View billing + Stripe Customer Portal at `/org/billing` - See dashboard with token-usage + MTD cost panel - See every program in the org (whether they own it or not — Inferred from RLS pattern) **Pages they access:** the full `/org/*` subtree, the `/admin/*` subtree is super-admin-only. **Subscription / seat restrictions:** pulls from the org's `plans` row — `instructor_seats`, `learner_seats`, `monthly_token_budget`, `features`.

### Instructor (Creator)

**What they can do:** - Create / edit / duplicate / delete their own programs (Chatrails) - Author bot nodes; configure system prompts, models, KB, rubrics - Invite learners + co-instructors + TAs - Grade transcripts - Build cert templates + issue cert awards (gated by plan `certificates: true`) - Use AI Chatrail Generator **Pages:** `/programs`, `/programs/new`, `/programs/generate`, `/programs/[id]/*`, `/dashboard`, `/rubrics`, `/account`. **Subscription restrictions:** plan-feature-gated via `lib/billing/gate.ts` — `canCreateProgram`, `canCreateChatbotNode`, `canIssueCertificate`, `canSeatLearner`, `canSeatInstructor`, `allowedModelsForPlan`. Free plan caps at 1 program / 3 chat nodes / 5 learners / no certs.

### Teaching Assistant (TA)

**What they can do:** - Co-grade with the instructor - Read and act on submissions for the program(s) they're attached to - See the gradebook, the bot nodes, the roster - Cannot create or delete programs (Inferred — author actions check `instructor | org_admin | super_admin` only) **Pages:** Likely the same as instructor for the programs they're attached to via `program_instructors.capacity = 'ta'`.

### Learner / Student

**What they can do:** - See programs they're enrolled in via `/learn` - Walk a journey at `/learn/[programId]` - Chat with bot nodes at `/learn/[programId]/[nodeId]` - Submit, view grades, view rubric breakdown - Earn certificates and download PDFs from `/learn/certificates`
**Cannot do:** see system prompts (RLS on `chatbot_configs` is staff-only; learners read a stripped-down `chatbot_learner_configs` view), see other learners' submissions, see draft/archived Chatrails.

### Public visitor

- Marketing pages: `/`, `/features`, `/pricing`, `/for-education`, `/for-corporate`, `/aup`, `/privacy`, `/terms`
- Cert verification: `/verify-cert/[code]`
- Auth: `/login`, `/signup`, `/forgot-password`, `/accept-invite/[token]`
- Cannot access any `/dashboard`, `/programs`, `/learn`, `/org`, `/admin` route — middleware bounces to `/login`

### Demo user

**Status:** seeded test users exist (super admin, org admin, instructor, TA, learner per `supabase/seed.sql`) but there's no separate "demo" role. The seed file is dev-only and not safe to run on production.

### Toggle / role-switching behavior

The header has a role switcher that swaps the "active" view among the orgs/roles a user belongs to via a cookie override (per `lib/auth/active-role.ts`). A user with multiple memberships sees both views and can context-switch without re-logging-in.

---

---

## 10. User Journey

### Instructor / Creator

1. **First visit** → marketing landing → `/pricing` → click "Free" or self-serve plan
2. **Signup** at `/signup` with intent = "Instructor / Trainer / SME" → auto-creates a personal org
3. **Onboarding** lands them on the instructor dashboard with a 5-step **OnboardingChecklist**: create program → add node → upload KB → invite learner → get first submission
4. **Authoring path A (blank):** `/programs/new` → bot node editor → KB upload → publish

5. **Authoring path B (AI):** `/programs/generate` → describe in plain English → 15–40s LLM call → land on Chatrail in DRAFT → edit each bot inline (PREVIEW CHAT panel) → publish
6. **Roster:** `/programs/[id]/roster` → invite by email or CSV → copyable invite-link panel for unreliable email
7. **Active phase:** dashboard cassettes show real PROGRESS / COMPLETE / N CERTS / LAST · 5m ago per program
8. **Grading:** `/programs/[id]/gradebook` → filter PENDING → click cell → grader panel slides in → AI suggest → adjust per-criterion → SAVE → learner notified
9. **Lifecycle:** PUBLISH / UNPUBLISH / ARCHIVE / RESTORE controls in overview → DELETE CHATRAIL with title-typed confirmation when abandoned
10. **Billing:** `/org/billing` → Stripe Checkout → upgrade if hitting caps; Customer Portal for plan / card management

## Learner / Student

1. **Receive invite** → email with link OR direct add (if instructor's account was already in the system)
2. **Accept invite** at `/accept-invite/[token]` → either signup (logged out) or claim-for-current-account (logged in, just shipped) → friendly mismatch path if email differs
3. **Land on `/learn`** → cassettes for each program with progress, due dates, last activity, cert count
4. **Open a Chatrail** at `/learn/[programId]` → progress bar + NEXT STEP banner + node grid with status badges (DONE / ACTIVE / AVAILABLE / LOCKED / RETRY) + ATTEMPTS · X/Y
5. **Enter a node** → for bots: chat screen with path rail (with LOCKED awareness), attempt counter, token meter, instructions strip, BRIEFING chip, SEND/STOP
6. **Submit** → confirmation → submitted banner → bounce back to journey
7. **View grade** → `/learn/[programId]/[nodeId]/grade` → rubric breakdown, instructor comment, AI summary if shared, retry / review / download transcript / verify certificate
8. **Earn certificate** → `/learn/certificates` → download PDF + public verification URL
9. **Renewal / cancellation:** for learner-paid programs, Stripe-managed; for invited learners, instructor controls

## Org admin

1. **Signup** as `org_admin` intent or **claim** an admin invite from someone else's org
2. **Dashboard** at `/dashboard` → token usage + MTD cost panel + Chatrails grid + members button
3. **`/org/members`** → manage roles, suspend / reactivate
4. **`/org/billing`** → Stripe Checkout / Customer Portal for plan changes

5. `/org/cert-template` → customize signer, body text, logo upload
6. **Active phase** → review usage week to week, escalate to higher plan if approaching cap
7. **Renewal:** Stripe handles subscription billing; webhook syncs `plan_code` and `subscriptions.status`; failed payment moves org to `past_due` and freezes new enrollments while preserving existing work

### Super admin

1. Signed up the platform first (manual SQL flag); subsequently lands on the super-admin dashboard
2. Browses `/admin/orgs`, `/admin/users`, `/admin/subscriptions`, `/admin/usage`
3. Manual seat-count / plan / flag overrides as needed
4. Reviews the audit ledger (UI planned, raw rows queryable via SQL)

### Public visitor / unauthenticated buyer

1. Lands on `/` → reads pillar features → clicks `/pricing` → reads the live plan rows
2. Clicks **GET STARTED** → `/signup` → onboarding
3. Or clicks `for-education` / `for-corporate` for segment-specific framing
4. Or visits `/verify-cert/[code]` to confirm a certificate someone showed them

---

## 11. Technical Stack

| LAYER | CHOICE |
|---|---|
| **Frontend framework** | Next.js 14 (App Router) + TypeScript + React 18 |
| **Styling** | Tailwind CSS + custom brutalist CSS layer ( `styles/brutalist.css` ) |
| **Component primitives** | Custom `components/brutalist/` + Radix UI (Dialog, Dropdown, Popover, Tabs, Toast) + lucide-react icons |
| **Forms / validation** | react-hook-form + Zod |
| **Visual graph editor** | `@xyflow/react` (React Flow) |
| **Rich text editor** | TipTap (starter-kit + image + link extensions) |
| **Backend** | Next.js Route Handlers + Server Actions; runs on Netlify Lambdas |
| **Database** | Supabase Postgres (project ref `qucizmsdoswunfsnqjam` ) |
| **ORM / DB tooling** | Drizzle ORM + drizzle-kit; Supabase client for RLS-aware queries; `postgres` for direct connections (used in pricing seed) |
| **Auth** | Supabase Auth (email + password, server-side cookie session via `@supabase/ssr` ) |
| **File storage** | Supabase Storage — buckets: `kb-files` , `node-files` , `certificates` , `org-logos` |
| **Vector search** | pgvector extension; `match_embeddings` Postgres RPC |
| **AI providers** | Anthropic ( `@anthropic-ai/sdk` v0.32), OpenAI ( `openai` v4.73), Google ( `@google/generative-ai` v0.21) — all behind `lib/llm/provider.ts` |
| **Embeddings** | OpenAI `text-embedding-3-small` (1536-dim native) OR Gemini `text-embedding-004` (768-dim padded to 1536) |
| **Streaming** | Server-Sent Events (SSE) in Route Handlers |
| **Payments** | Stripe ( `stripe` v17, `@stripe/stripe-js` via wrapper) |
| **Hosting** | Netlify ( `@netlify/plugin-nextjs` ) |
| **Email** | Resend (optional — falls back to console log) |
| **Rate limiting** | Upstash Redis ( `@upstash/ratelimit` + `@upstash/redis` ) with in-memory fallback |
| **Monitoring** | Sentry ( `@sentry/nextjs` ) |

| LAYER | CHOICE |
|---|---|
| **PDF generation** | `@react-pdf/renderer` |
| **Document parsing** | `pdf-parse` (PDFs), `mammoth` (DOCX, partial) |
| **Charts** | Recharts |
| **Notifications** | sonner (toasts) |
| **Testing** | Vitest + GitHub Actions CI |
| **CI/CD** | GitHub Actions for typecheck/lint/test/build; Netlify auto-deploy on push to `main` |
| **Bundle analysis** | `@next/bundle-analyzer` (opt-in via `npm run analyze`) |

## Integrations not yet present (planned)

- LTI 1.3 (Canvas / Blackboard / Moodle) — Unknown / Not Built
- SSO (SAML / OIDC) — gated to `org_school+` plans, IdP integration code not present
- SCORM / xAPI export — not built
- PostHog / Plausible product analytics — Unknown (mentioned in ROADMAP pre-work as optional)

## Node version

`engines.node: ">=20"` enforced in `package.json`.

---

# 12. Architecture Overview

## Folder structure

```
app/
  (marketing)/           public — landing, features, pricing, for-education,
for-corporate, /privacy, /terms, /aup, /docs
  (auth)/                sign in / up / forgot-password / accept-invite /
verify
  (app)/                 authenticated app, role-aware
    dashboard/           dispatcher → instructor.tsx | learner.tsx | org-
admin.tsx | super-admin
    programs/            list / new / generate / [id]/
{nodes,kb,roster,gradebook,settings,builder,analytics}
    learn/               learner journey + chat + grade view + certificates
    rubrics/             rubric library + editor
    org/                 members, billing, settings, cert-template
    admin/               super admin: orgs, users, subscriptions, usage
    notifications/       in-app notification feed
    account/             user account + export + delete
    seats/               seat allocation page (Inferred)
    kb/                  cross-Chatrail KB hub
  api/
    chat/stream/         SSE chat streaming
    chat/preview/        ephemeral preview chat (staff-only, no DB writes)
    grade/suggest/       AI rubric scoring suggestion
    auth/callback/       Supabase OAuth-style callback
    certificates/[id]/pdf/  PDF stream
    conversations/[id]/export/  markdown transcript export
    programs/[id]/gradebook/csv/  CSV export
    account/export/      data export
    account/delete/      account deletion
    stripe/checkout/     Stripe Checkout session creation
    stripe/webhook/      Stripe webhook handler
    stripe/portal/       Customer Portal redirect
    health/              /api/health probe

components/
  brutalist/             Cassette, Chip, Btn, Icon, Eyebrow, Frame, Skeleton,
IconBtn — design primitives
  shell/                 Header, Footer, role switcher
  builder/               React Flow nodes + inspector
  chat/                  ChatScreen
  gradebook/             table + grader-panel (slide-out)
  dashboard/             OnboardingChecklist
  learn/                 non-bot-node renderers, slides viewer

lib/
  db/                    Drizzle schema + types
  supabase/              server / client / middleware / service-role helpers
  auth/                  RBAC + active-role with cookie override
```

```
  llm/                    provider adapter, cost calc, usage logger, prompt
builder, grading helpers, generate-chatrail
  rag/                    chunker, extractor, indexer, vector search
  path/                   pure path-progress engine + actions
  billing/                plan-feature gates
  stripe/                 client + plans
  email/                  Resend client + templates
  notifications/          createNotification helper
  ratelimit.ts            Upstash with in-memory fallback
  observability/          logger
  utils/                  cx, binary index, relative-time

supabase/
  migrations/             13 SQL files (extensions → identity → curriculum →
ai → conversations → grading → certificates → billing → policies → security →
perf-indexes → cert-templates → notifications)
  seed.sql                one of every role + a graded submission

styles/
  brutalist.css           ported from prototype's styles.css verbatim
  globals.css             Tailwind base + brutalist import

tests/                    vitest — path-progress, chunker, relative-time,
generate-chatrail-extract (19 tests)
```

## Major app areas (top-level routes)

- `(marketing)` — public-facing
- `(auth)` — auth flows
- `(app)` — gated by middleware that requires a Supabase session
- `api/` — Route Handlers for streaming, exports, Stripe, health

## Important routes

- `/programs/generate` — AI Chatrail generator (just shipped)
- `/programs/[id]/builder` — React Flow visual editor
- `/programs/[id]/nodes/[nodeId]` — bot node inspector with inline preview chat
- `/learn/[programId]/[nodeId]` — learner chat (also handles content / pdf / link / milestone / cert via dispatch)
- `/api/chat/stream` — SSE streaming endpoint, server-only LLM keys
- `/api/chat/preview` — staff-only ephemeral preview, no DB writes
- `/verify-cert/[code]` — public certificate verification

## Authentication flow

1. User signs up via `/signup` → `supabase.auth.signUp()` returns user_id
2. A trigger in `00010000000002_identity.sql` inserts a row into `public.users` mirroring `auth.users`
3. Server actions in `app/(auth)/actions.ts` provision org membership based on intent (instructor/org_admin/learner/ta) or invite token
4. Middleware ( `middleware.ts` ) reads the Supabase cookie session on every gated request
5. `lib/auth/active-role.ts` resolves the active org + role from cookie + memberships
6. RBAC checks in server actions gate sensitive ops ( `requireSessionUser` , `getActiveRole` , role allow-lists)
7. RLS policies in `00010000000009_policies.sql` (and `00010000000010_phase_a_security.sql` ) enforce row-level access at the DB layer

## Tenant / organization structure

- The canonical tenant is `organizations` ; everything is `organization_id` -scoped via FK
- Users belong to many orgs via `organization_members` with one role per (org, user)
- Programs belong to one org; nodes / configs / KB / conversations / submissions / grades cascade off the program
- Cross-org actions are blocked at the gate AND at RLS (defense in depth)

## How data moves

- **Authoring:** server action → service role insert into `programs` / `path_nodes` / `chatbot_configs` → revalidate
- **Chat:** client POST → SSE route → admin client (RLS-bypass) reads bot config + KB + history → calls LLM provider (server-only key) → streams SSE deltas → on completion persists assistant message + updates conversation token totals + writes `usage_logs` row
- **Grading:** client opens grader panel → client supabase reads transcript (RLS-allowed for staff) → client POSTs to `/api/grade/suggest` → server uses admin client to read rubric + criteria + transcript → calls `suggestRubricScore` LLM helper → returns JSON → instructor edits + saves grade via server action → action mirrors status to `conversations` , runs `maybeAwardCertificates` , fires a `grade_returned` notification
- **Cert award:** server-side check on every grade save + mark-complete → if criteria met, insert `certificate_awards` row → email learner + create notification → learner downloads PDF from `/api/certificates/[id]/pdf?v=[verification_code]`

---

## 13. Data Model

All tables are in the `public` schema; RLS is enabled on every tenant table.

### Identity ( `00010000000002_identity.sql` )

- **`users`** — profile mirroring `auth.users`. `is_super_admin` boolean.
- **`organizations`** — tenant root. `slug`, `org_type`, `plan_code`, `stripe_customer_id`.
- **`organization_members`** — (org, user, role) — many-to-many with role per relationship.
- **`invites`** — email + token + role + program_id (optional). Status: pending / accepted / revoked / expired.
- **`csv_imports`** — record of bulk CSV imports.
- **`audit_logs`** — actor / action / target / metadata; written by sensitive server actions.

### Curriculum ( `00010000000003_curriculum.sql` )

- **`programs`** — top-level Chatrail. `title`, `description`, `status` (draft/published/archived), `default_model`, `passing_threshold`, `monthly_token_budget`, `learner_pays`, `learner_price_cents`, `share_conversations_with_org_admin`.
- **`program_instructors`** — (program, user, capacity) where capacity ∈ owner / co_instructor / ta.
- **`program_enrollments`** — (program, user) with `status` (active / suspended).
- **`path_nodes`** — every node. `type` (bot / content / pdf / slides / link / milestone / cert), `title`, `display_order`, `points`, `available_at`, `due_at`, `is_required`, `config` jsonb (type-specific).
- **`path_edges`** — (source, target, condition jsonb) with kinds: `null` (default — source must be completed), `after`, `min_score`, `date`, `either`.
- **`node_rules`** — extra release rules per node (open_on_date, after_prereq, min_score, either_or, branching, skip).

### AI ( `00010000000004_ai.sql` )

- **`chatbot_configs`** — one per bot node. `system_prompt`, `learner_instructions`, `model`, `temperature`, `max_tokens`, `token_budget`, `attempts_allowed`, `rubric_id`, `ai_grading_enabled`, `use_program_kb`, `node_kb_id`. Plus `bot_name`, `avatar_initials`, `conversation_goal`, `completion_criteria`.
- **`knowledge_collections`** — program-scoped (one default per program created automatically).
- **`knowledge_files`** — uploaded file refs + `storage_path` + `status` (uploading / parsing / chunking / embedding / indexed / failed).

- **`document_chunks`** — text + page + offset metadata.
- **`embeddings`** — `vector(1536)` pgvector column.

A learner-safe **view** `chatbot_learner_configs` excludes `system_prompt`, `conversation_goal`, `completion_criteria` (added in Phase A security migration).

### Conversations (`00010000000005_conversations.sql`)

- **`conversations`** — one per (learner, node, attempt). `status` (not_started / in_progress / submitted / graded / completed / needs_revision), `total_prompt_tokens`, `total_completion_tokens`, `ai_summary`, `submitted_at`.
- **`conversation_messages`** — role + content + per-message token counts + citations jsonb + model.
- **`submissions`** — snapshot at submit time. `delivery_status` (on_time / late), FK to conversation + program + node + learner + org.
- **`analytics_events`** — generic event log (Inferred — schema-only; usage may be partial).

### Grading (`00010000000006_grading.sql`)

- **`rubrics`** — name + description + total_points + is_visible_to_learners.
- **`rubric_criteria`** — name + description + max_points + display_order.
- **`rubric_levels`** — label + points + description + display_order (per criterion).
- **`grades`** — one per submission. `status` (pending_review / graded / needs_revision / excused / not_submitted), `score`, `max_score`, `percentage`, `instructor_comment`, `ai_summary`, `ai_suggested_score`, `rubric_id`, `graded_at`, `graded_by`.
- **`rubric_scores`** — per-criterion: grade_id + criterion_id + score + comment.

### Certificates (`00010000000007_certificates.sql`)

- **`certificate_templates`** — per org. Signer name, body text, logo URL, paper size.
- **`certificates`** — definition attached to a node (which milestone triggers issuance, with min grade %).
- **`certificate_awards`** — one per (learner, program, certificate). `verification_code` (unique), `awarded_at`. PDF generated on demand from `/api/certificates/[id]/pdf`.

### Billing (`00010000000008_billing.sql`)

- **`plans`** — code, name, scope (instructor / organization), monthly_price_cents, instructor_seats, learner_seats, monthly_token_budget, features jsonb.
- **`subscriptions`** — Stripe customer + sub IDs, status (trialing / active / past_due / canceled / unpaid / incomplete), current_period_end.

- **`seat_pools`** — org-owned or instructor-owned seat banks (Inferred — schema only, full UX is roadmap).
- **`seat_assignments`** — link seat to user + program (Inferred).
- **`billing_events`** — Stripe webhook event log.
- **`usage_logs`** — one row per LLM call. organization_id, program_id, node_id, conversation_id, user_id, kind (chat / chat-preview / embedding / grade_suggest), model, prompt_tokens, completion_tokens, est_cost_usd.

### Notifications (`00010000000040_phase_t_notifications.sql`)

- **`notifications`** — user-scoped. Kind (invite_received / grade_returned / cert_awarded / etc.), title, body, href, metadata jsonb, read_at.

### Other

- **Indexes:** Phase H added 5 targeted indexes for gradebook + analytics joins (`00010000000020_phase_h_perf_indexes.sql`).
- **Default cert templates:** seeded in `00010000000030_phase_s_default_cert_templates.sql`.

### Entity relationship summary

```
organizations 1–N  organization_members  N–1 users
organizations 1–N  programs              1–N path_nodes
                                         1–N path_edges
                                         1–N program_enrollments  N–1 users
                                         1–N program_instructors  N–1 users
                                         1–N knowledge_collections 1–N
knowledge_files 1–N document_chunks 1–1 embeddings
path_nodes (type=bot) 1–1 chatbot_configs
chatbot_configs N–1 rubrics 1–N rubric_criteria 1–N rubric_levels
programs 1–N conversations 1–N conversation_messages
conversations 1–1 submissions 1–1 grades 1–N rubric_scores
programs 1–N certificates 1–N certificate_awards
organizations 1–1 subscriptions  N–1 plans
```

---

## 14. AI Usage

### Where AI is used

1. **Chat streaming** ( `/api/chat/stream` ) — the primary product surface; learner ↔ bot conversations
2. **Preview chat** ( `/api/chat/preview` ) — staff-only, ephemeral, no DB writes
3. **AI grading suggestion** ( `/api/grade/suggest` ) — summarizes transcript + suggests per-criterion rubric scores; instructor reviews + overrides
4. **AI Chatrail generator** ( `/programs/generate` + server action) — creator brief → multi-node Chatrail with prewritten system prompts and learner instructions
5. **Embeddings** — KB documents are chunked, embedded (OpenAI 1536 or Gemini 768→1536 padded), stored in pgvector, retrieved on every chat turn (when `use_program_kb=true` )

### Model providers used / planned

- **Anthropic Claude** — `claude-haiku-4-5` (default), `claude-sonnet-4-6` , `claude-opus-4-7` , `claude-3-5-sonnet-latest` , `claude-3-5-haiku-latest`
- **OpenAI** — `gpt-4o` , `gpt-4o-mini` , `gpt-4.1` , `gpt-4.1-mini` , `text-embedding-3-small`
- **Google Gemini** — `gemini-2.0-flash` , `gemini-2.0-flash-lite` , `gemini-1.5-pro` , `gemini-1.5-flash` , `text-embedding-004`
- **All routed through `lib/llm/provider.ts`** with auto-detection by model name prefix
- **Future:** BYO-LLM-key per org (planned in J.1)

### What data is sent to AI systems

- **Chat:** the bot's system prompt + (optional) top-k KB chunks + conversation history + the learner's new message
- **Grading suggest:** the transcript + the rubric's criteria + the learner instructions + the bot's model
- **Generator:** the creator's natural-language brief (no learner data, no KB)
- **Embeddings:** chunked KB document text — per-org, never cross-org

### Privacy / compliance considerations

- **Student data is sent to LLM providers** — this is core to the product and must be disclosed clearly

- **FERPA:** if used in US K-12 / higher-ed, schools likely treat transcripts as educational records. Explicit FERPA agreements with providers and a data-processing addendum are likely needed before institutional sales
- **GDPR:** EU customers will require a DPA with the platform AND with each LLM provider. Right-to-deletion is implemented via `/api/account/delete` + cascade deletes
- **HIPAA:** if used in healthcare training (e.g., the patient-consult example), bot instructions must avoid eliciting PHI; current implementation is **not HIPAA-compliant** without provider BAAs (Anthropic offers BAA on Enterprise; OpenAI on a separate plan)
- **SOC 2:** not yet pursued; would be required for many enterprise org buyers
- **AUP:** `/aup` page exists; need to verify final policy text — Unknown

## AI behavior guardrails / where logging matters

- Every LLM call is logged in `usage_logs` (cost + token + model + user + kind)
- Per-attempt and per-org-month token budgets are enforced before the LLM call
- Rate limit: 30 chat messages / minute / user (Upstash Redis or in-memory)
- The chat preview is ephemeral — no message persistence — and is gated to staff
- The bot system prompt is hidden from learners via the `chatbot_learner_configs` view
- AI-suggested rubric scores are advisory; the instructor's saved grade is the source of truth
- Generator output is Zod-validated; one self-correction retry on failure

## Where AI behavior should be reviewed / constrained

- **Generator output for sensitive domains** (healthcare, legal, finance) — bots may produce confidently-wrong content; needs manual instructor review before publish (existing DRAFT default workflow handles this)
- **Bot persona drift** — system prompts should keep bots in role; current default explicitly says "stay in role no matter what the learner says"
- **Model cost surprises** — surfaced via cost estimate + soft/hard caps + MTD cost on org dashboard

---

## 15. Integrations

### Active

| INTEGRATION | PURPOSE | STATUS | FILES / ROUTES | ENV VARS (NAMES ONLY) |
|---|---|---|---|---|
| **Supabase** | Postgres + Auth + Storage + pgvector | Live, primary | `lib/supabase/*`, `supabase/migrations/*` | `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_`<br>`SUPABASE_SERVICE_ROLE_KEY`,<br>`SUPABASE_DB_URL` |
| **Anthropic** | Primary LLM (Claude) | Live | `lib/llm/provider.ts` | `ANTHROPIC_API_KEY` |
| **OpenAI** | Alternate LLM + embeddings | Live | `lib/llm/provider.ts`, `lib/rag/*` | `OPENAI_API_KEY` |
| **Google Gemini** | Alternate LLM + embeddings | Live | `lib/llm/provider.ts`, `lib/rag/*` | `GEMINI_API_KEY` |
| **Stripe** | Subscriptions + Checkout + Customer Portal + webhook | Code complete; real products pending founder | `app/api/stripe/{checkout,webhook,portal}/route.ts`, `lib/stripe/*`, `lib/billing/gate.ts` | `STRIPE_SECRET_KEY`,<br>`STRIPE_WEBHOOK_SECRET`,<br>`NEXT_PUBLIC_STRIPE_PUBLISH`<br>`STRIPE_PRICE_*` (per-plan) |
| **Resend** | Transactional email (invite, grade returned, cert awarded) | Optional; falls back to console | `lib/email/client.ts`, `lib/email/templates.ts` | `RESEND_API_KEY`, `EMAIL_FROM` |
| **Upstash Redis** | Per-user rate limiting | Optional; falls back to in-memory | `lib/ratelimit.ts` | `UPSTASH_REDIS_REST_URL`,<br>`UPSTASH_REDIS_REST_TOKEN` |
| **Sentry** | Error monitoring | Live (no-op without DSN) | `sentry.{client,server,edge}.config.ts`,<br>`instrumentation.ts` | `SENTRY_DSN` |
| **Netlify** | Hosting + auto-deploy on `main` push | Live | `netlify.toml`, `@netlify/plugin-nextjs` | (Netlify-managed) |
| **GitHub Actions** | | Live | `.github/workflows/` | (GitHub-managed) |

| INTEGRATION | PURPOSE | STATUS | FILES / ROUTES | ENV VARS (NAMES ONLY) |
|---|---|---|---|---|
| | CI (typecheck → lint → test → build) | | | |

### Planned but not yet wired

| INTEGRATION | PURPOSE | STATUS |
|---|---|---|
| **LTI 1.3** (Canvas / Moodle / Blackboard) | LMS interop for institutional buyers | Not built |
| **SAML / OIDC SSO** | Enterprise SSO (gated to `org_school+` ) | Plan flag exists; IdP integration not present |
| **PostHog / Plausible** | Product analytics | Not wired (mentioned in ROADMAP pre-work) |
| **DOCX upload** for KB | Beyond PDF / TXT / MD / CSV | Library installed ( `mammoth` ), full pipeline incomplete |
| **Custom domain** | `chatquest.app` (vs `chattrail.netlify.app` ) | DNS / Netlify config Unknown |

### Webhooks that exist

- `/api/stripe/webhook` — `checkout.session.completed` , `customer.subscription.updated` , `customer.subscription.deleted` , `invoice.payment_failed` (Inferred from CHANGELOG description)

---

---

## 16. Security, Privacy, and Compliance

### Sensitive data handled

- User identity (email, full name, optional avatar)
- Auth credentials (managed by Supabase Auth; passwords never visible to app code)
- Conversation transcripts (educational content; potentially personally identifying or sensitive depending on what learners write)
- Submission scores + grades + instructor comments
- Certificate awards with unique verification codes
- Billing data (Stripe customer ID, subscription state — never card data, which stays in Stripe)

- Knowledge base uploads (whatever the instructor uploads — could be IP-sensitive, personal, or institutional)
- Org logos + signatures

## Authentication and authorization

- **Authentication:** Supabase Auth, email + password, server-side cookie session via `@supabase/ssr`
- **Email verification:** disabled in dev ( `supabase/config.toml` ); should be enabled for prod (BLOCKED ON YOU per ROADMAP)
- **MFA:** Supabase supports it but UI integration is **Unknown**
- **Authorization:** three layers — (1) middleware route gates, (2) server-action role checks via `getActiveRole` , (3) Postgres RLS policies ( `00010000000009_policies.sql` + `00010000000010_phase_a_security.sql` )
- **RBAC:** five roles ( `super_admin | org_admin | instructor | ta | learner` ) with org-scoped membership

## Role-based access control

- Cross-org access blocked at gate AND RLS (defense in depth)
- Learners cannot read bot system prompts (separate `chatbot_learner_configs` view)
- Staff (instructor / TA / org_admin in the org, or super_admin) bypass learner-only guards
- Permanent delete requires title-typed confirmation
- Sensitive server actions write `audit_logs` rows

## Payment data handling

- Card data never touches Chatrail servers — handled entirely by Stripe Checkout / Customer Portal
- Stripe customer + subscription IDs stored on `organizations` and `subscriptions`
- Webhook signature verification implemented in `/api/stripe/webhook`

## Compliance posture

- **FERPA** (US educational records): not yet formally compliant; institutional sales need DPAs + provider BAAs. Inferred — no FERPA addendum present.
- **GDPR** (EU): right-to-deletion via `/api/account/delete` + cascade FKs; right-to-export via `/api/account/export` . Full DPA wording in `/privacy` should be reviewed by counsel before EU sales.
- **HIPAA** (health data): **not compliant** without provider BAAs; should not be used for PHI today.

- **SOC 2:** not pursued.
- **COPPA** (under-13 US): not addressed; Chatrail should not be used by under-13s without parental consent + institutional process.

### Security gaps / unknowns

- **Email confirmation off in dev** — BLOCKED ON YOU to enable for prod
- **Custom domain + DNS / SPF / DKIM / DMARC** — not yet set up
- **Bug bounty / vuln-disclosure** — only `security@chatquest.app` in `SECURITY.md`; mailbox status Unknown
- **Service-role key in `.env.local`** — common dev pattern but should be rotated on deployment per ROADMAP
- **Audit log viewer UI** — rows are written but the UI is Unknown / planned
- **MFA enforcement** — Supabase supports it; not enforced

### Recommended improvements (security)

1. Enable Supabase email confirmation for production
2. Set up DKIM + SPF + DMARC on the sending domain
3. Get a custom domain + valid TLS chain
4. Rotate the seed-test service role key
5. Build the `/admin/audit` viewer
6. Get DPAs in place with each LLM provider before institutional sales
7. Pursue SOC 2 Type 1 once revenue justifies (~$30K initial)
8. Add MFA enforcement for `org_admin` and `super_admin` roles
9. Add a CSP that's enforcing (currently report-only per CHANGELOG Phase A)

---

## 17. Current Development Status

### Complete

- Phase 1 MVP end-to-end workflow (workflow 1)
- Phases A–I per CHANGELOG (production safety, branded emails, visual builder, certificates, Stripe code, cost control, hygiene, UX, performance, marketing/SEO)
- This session's work: lifecycle controls, visibility data, attempts handling, stop button, rubric edit, generator, KB toggle, AI-grading toggle, claim-invite, duplicate rubric, cost estimate, org

token panel, attempts indicator, programs filter, /learn richness, gradebook correctness fix, etc.
- 19 vitest tests, all passing
- TypeScript clean ( `tsc --noEmit` )
- GitHub Actions CI green
- Brutalist design system fully ported

## Partially built

- **Stripe monetization** — code paths complete; real Stripe products + price IDs pending founder action
- **Email delivery** — code paths complete; `RESEND_API_KEY` pending founder action; DNS records pending
- **Audit log viewer** — rows are written; UI roadmapped (J.3)
- **DOCX KB ingestion** — library installed; pipeline not finished
- **Seat pools / seat assignments** — schema present; full allocation UX likely partial (per ROADMAP item D)
- **Notifications UI** — schema + helper exist; bell badge + feed page exist (Inferred from `app/(app)/notifications/` ); breadth-of-event coverage Unknown

## Placeholder

- Custom domain `chatquest.app` (mentioned but Unknown if registered)
- BYO-LLM-key (J.1 placeholder for future)
- LTI / SCORM / xAPI integrations
- SAML / OIDC SSO (gate exists, IdP code does not)

## Broken / needs attention

- **None visibly broken** — last full typecheck + tests + manual smoke test were green
- The repo's package name is still `chatquest` while the product is `Chatrail` — Inferred non-blocking

## Needs testing

- Stripe webhook flow end-to-end against real test-mode products (once founder sets them up)
- Multi-org user role-switching (manual testing only so far)
- AI generator output across edge cases (multilingual prompts, very long briefs, refusing to comply briefs) — test fixtures cover schema but not behavior
- Rate limiting at Upstash (in-memory works locally)

- Sentry error capture (DSN-dependent)
- Email deliverability with Resend (no DNS yet)
- HTML safety of `bodyMarkdown` content nodes generated by AI (escaped server-side; review)

### Needs design cleanup

- Mobile responsiveness — Phase G addressed major surfaces but path builder is documented as needing an "outline view" on mobile
- Empty states across less-traveled pages (Inferred)
- Marketing pages could expand "for-education" / "for-corporate" with case studies once pilots exist

### Needs backend work

- Notifications email digesting (right now it's in-app only)
- Email templates for cert-awarded path (Inferred — invites + grade-returned exist; cert template Unknown)
- Background KB indexing (currently inline; Phase H notes background-queue improvement)

### Needs deployment / ops work

- Resend domain + DNS
- Custom domain
- Supabase site URL / redirect URL config for prod
- Stripe products + price IDs
- Sentry DSN
- Optional Upstash + PostHog credentials
- Service-role key rotation
- MFA on Supabase / Netlify / GitHub accounts (BLOCKED ON YOU)

---

---

## 18. Known Issues and Risks

### Technical risks

- **LLM provider concentration** — Anthropic outage, Gemini outage, or OpenAI outage each take down a model class; the multi-provider adapter mitigates this but per-bot model choice is still single-vendor

- **Postgres single-region** — Supabase single-region; latency for distant regions; backup / DR Unknown
- **Netlify Lambda limits** — 26s default timeout could clip slow LLM calls (the generator can take 15–40s); need to verify Netlify function timeout config
- **Direct Postgres flakiness on Lambda** — already burned us once on `/pricing`; pgvector RPC fix was Phase A; future Drizzle migrations should avoid relying on outbound :5432 from Lambdas
- **No background job queue** — KB indexing, email digests, big exports run inline today

## Business risks

- **AI tutoring is a hot, crowded category** — fast-moving competition from Anthropic / OpenAI / Google directly, plus countless YC startups. Differentiation needs to be lived, not just claimed.
- **Buyer persona is split** between solo creators (low ACV, high churn) and institutions (high ACV, slow sales cycles). Pricing reflects both but the GTM motion is two motions, not one.
- **K-12 sales** require trust signals (FERPA, COPPA, accessibility audits, district procurement) Chatrail does not yet have.
- **Higher-ed sales** require LTI integration with Canvas / Blackboard / Moodle (not built).
- **Token cost in COGS** — at high volume, LLM cost is most of variable cost; misuse / cheat-prompts could shred margin without strict caps.

## UX risks

- Path builder on mobile is acknowledged-weak
- The AI generator's 15–40s wait is brittle perception-wise even with progress messages — streaming the output would feel snappier (not yet done)
- Brutalist design is polarizing; might need a softer variant for institutional buyers (Inferred — no theming exists)

## Security risks

- Email confirmation off in dev (must enable for prod)
- Service-role key has been pasted to chat per ROADMAP — must be rotated
- CSP is report-only — needs to flip to enforcing
- No MFA enforcement for admins
- Audit log viewer UI not built — investigations require SQL

## Legal / compliance risks

- FERPA / GDPR / HIPAA posture is informal; institutional sales blocked until DPAs + BAAs in place
- AUP / Terms / Privacy text is scaffold — needs counsel review
- AI training opt-out / data-residency claims with each LLM provider need formal documentation before enterprise sales
- DMCA / takedown process not formalized
- Accessibility audit (WCAG 2.1 AA) not yet done — many institutions require it

## Market risks

- Anthropic / OpenAI could ship competing first-party LMS surfaces
- Khan Academy / Coursera could launch chatbot-native modes
- Existing LMS incumbents (Canvas / Blackboard) could close the gap with bolt-ons that are "good enough" for the median buyer

## Cost risks

- LLM token cost per learner per Chatrail is the biggest variable cost; if the generator + preview + grading + chat all use Sonnet, a single learner attempt can hit $0.50+. At 10K-learner cohorts, this is real money.
- Free tier (50K tokens) abusers could chain accounts to grind on the free product — partially mitigated by rate limits, but not abuse detection

## AI usage risks

- Bots producing confidently-wrong content in sensitive domains (legal, medical, financial) without instructor review
- Prompt injection via learner messages → bot reveals system prompt / changes role → mitigated by "stay in role" defaults but not adversarially tested
- Learners using bots to do their thinking for them (the meta-risk for AI tutoring as a category) — partially mitigated by transcript grading, but a real conversation about academic integrity belongs in the AUP

## Code quality issues visible from the repo

- Some inline relative-time copies were extracted in this session, but the codebase still has duplication across cassette renderers (instructor dashboard / programs list / /learn) that could share more
- A few `unknown as` casts in places where the Supabase TS types don't match the joined select shape — fine but brittle if schema changes

- The legacy package name `chatquest` vs product name `Chatrail` is a minor branding inconsistency
- Some TypeScript linter warnings about exhaustive deps were silenced inline — Inferred

---

---

## 19. Roadmap

### Phase 1 — Stabilize current build

- [ ] Rotate service-role key + Gemini key (BLOCKED ON YOU)
- [ ] Enable Supabase email confirmation for prod (BLOCKED ON YOU)
- [ ] Set up Resend account + DNS records (BLOCKED ON YOU)
- [ ] Custom domain + SSL (BLOCKED ON YOU)
- [ ] Flip CSP from report-only to enforcing
- [ ] Build the `/admin/audit` viewer UI
- [ ] Background queue for slow KB indexing
- [ ] Streaming output for the AI generator (improves perceived perf)

### Phase 2 — Complete MVP

- [ ] DOCX KB upload pipeline
- [ ] Per-node attempt-count UI enforcement everywhere (started this session)
- [ ] Org-level KB collections (vs program-level only)
- [ ] BYO LLM key per org
- [ ] Cert email template for `cert_awarded` event
- [ ] Mobile-friendly path builder ("outline view")
- [ ] Accessibility audit + WCAG 2.1 AA fixes
- [ ] Tighten EmptyState / error states across less-traveled pages

### Phase 3 — Pilot or early user testing

- [ ] Recruit 3–5 pilot creators across segments (university, corporate L&D, solo coach, K-12, healthcare training)
- [ ] Onboarding interview → close-the-loop on activation friction
- [ ] Measure: time-to-first-published-Chatrail, time-to-first-learner-submission, time-to-first-graded-submission
- [ ] Add a "What changed since last visit" digest email

- [ ] Add NPS prompt at 30-day mark
- [ ] Capture testimonials + case studies for marketing

## Phase 4 — Monetization and sales readiness

- [ ] Stripe products + price IDs in test mode → live mode (BLOCKED ON YOU for setup; code is wired)
- [ ] Annual billing option (10–20% discount)
- [ ] Coupons / promo codes for pilots
- [ ] Customer Portal smoke test against real cards
- [ ] Failed-payment dunning emails
- [ ] DPAs with Anthropic, OpenAI, Google
- [ ] Counsel-reviewed Terms / Privacy / AUP
- [ ] Anthropic BAA if pursuing healthcare training segment
- [ ] FERPA addendum scaffold for K-12 / higher-ed
- [ ] Cohort-export workflow for IT departments
- [ ] Sales collateral: deck, one-pager, demo script

## Phase 5 — Scale / enterprise / growth

- [ ] LTI 1.3 launcher for Canvas / Moodle / Blackboard
- [ ] SAML / OIDC SSO (gated tier already exists)
- [ ] SCORM 1.2 / 2004 export of completion data
- [ ] xAPI statements
- [ ] SOC 2 Type 1
- [ ] Multi-region Supabase or read replicas for global cohorts
- [ ] Background queue (BullMQ / SQS / Inngest) for KB indexing + email digests + exports
- [ ] Multilingual UI (i18n)
- [ ] Marketplace / template library
- [ ] Partner-channel program (instructional design agencies, corporate L&D consultancies)
- [ ] Native mobile apps OR fast PWA
- [ ] Analytics deeper dives (mastery curves, common-misconception detection from transcript clustering)

---

## 20. Marketing and Positioning Notes

### Best elevator pitch (≤30s)

"*Chatrail is a chatbot-native learning management system. Instructors design AI tutor conversations the way you'd lay out a flowchart — each node is a bot with its own system prompt, knowledge base, and rubric — and learners chat their way through. Transcripts get graded with AI assistance, certificates auto-issue, and the whole thing is multi-tenant from day one. Think Canvas, but built for the era where conversation is the lesson, not the slide.*"

### Longer sales description

Chatrail is the LMS for organizations that have figured out their best teaching is conversational and want to scale that without losing rigor. Trainers, professors, coaches, and L&D teams use Chatrail's visual path builder to compose chatbot-driven learning paths — Socratic discussion bots, role-play simulations, debate opponents, examiners, reflective coaches — each with its own pedagogically-tuned system prompt and grading rubric. Learners enter, chat through every node, and submit; instructors grade transcripts against rubrics with AI-suggested per-criterion scores; verifiable certificates issue automatically. Multi-tenant from day one with seat-based billing through Stripe, plan-feature gating, organization-scoped knowledge bases, role-aware dashboards, and a confident brutalist visual identity that signals "this is not your enterprise LMS."

### Suggested homepage positioning (already partially in place)

Lead with: "*Chatbot-native LMS for serious learning.*" Subtitle: "*Build AI tutors. Wire them into a learning program. Grade transcripts with rubrics. Issue certificates. The brutalist way.*" Followed by 6 feature cassettes (already in code): VISUAL PATH BUILDER, CHATBOT NODES + RAG, TRANSCRIPT GRADEBOOK, CERTIFICATES, SEAT POOLS + STRIPE, … Then a "see it work" demo loop or recording. Then a "for educators / for corporate" two-card split linking to deeper segment pages. Pricing fed live from the `plans` table.

### Audience-specific pitches

- **Higher ed instructor:** "*Your seminar discussions don't scale. Chatrail does. Build a Socratic bot for your reading list, grade transcripts with the rubric you already wrote, and issue verifiable course-completion certificates.*"
- **Corporate L&D:** "*Compliance training that's actually practiced, not just clicked through. Build role-play bots for sales discovery calls, conflict mediation, security incident response — each with its own KB and cost cap. Seat-based pricing your finance team understands.*"
- **K-12 teacher / admin:** "*Differentiated instruction at the conversation level — every student gets a Socratic tutor for the topics they're struggling with, and you see where each cohort is stuck.*"

- **Coach / SME:** "*You've already templated your best 1:1 conversation a hundred times. Package it as a Chatrail and your expertise scales with zero marginal cost. Keep IP, keep brand, keep margin.*"

## Possible slogans / taglines

- "*The chatbot-native LMS.*"
- "*Conversation is the lesson.*"
- "*Teach by talking — at scale.*"
- "*Your AI tutor, on rails.*"
- "*Brutalist learning, built for AI.*"
- "*Chatbots that grade themselves. Almost.*"

## Demo script summary (≤5 min)

1. Open `/programs/generate` — type "*5-node Chatrail teaching Stoic philosophy with a Socratic discussion bot, a debate bot, a reflection coach, and a final assessment bot*"
2. Click GENERATE — wait while progress messages tick
3. Land on the Chatrail in DRAFT — show the cassette grid
4. Open one bot → show the system prompt the AI wrote → click PREVIEW CHAT → exchange a turn
5. Edit one phrase in the system prompt to demonstrate fine control
6. Show the live cost estimate update as we change the model
7. Switch tabs to show ROSTER → invite fictional learner via copyable link
8. Open `/learn` as the learner → enter the bot → chat → submit
9. Switch back to instructor → gradebook → click cell → AI suggest → adjust per-criterion → SAVE
10. Show certificate verification page with a unique code
11. Close on the org-admin dashboard's MTD cost panel

## Early customer outreach angle

- **For solo instructors:** "I built the LMS I wanted as a teacher. AI tutors with rubrics, transcripts you can grade, certificates that verify. Free tier with one program — let me know what you'd add." Personal-voice outreach in education Twitter / LinkedIn / Substacks.
- **For corporate L&D:** "Your role-play training is going to get an AI rebuild whether you want it to or not. Want to be early or late?" — short pilot offers (3 months, 1 program, 100 seats) at a token-cost-only price.

- **For instructional designers:** "We built the platform; you bring the curriculum design. Co-creation pilots with rev-share on the resulting marketplace." — long-term move toward template marketplace.

### Why this product is credible

- Phase 1 MVP is **shipped and runs on real LLM + Supabase + Netlify** — not a slideware demo
- 19 unit tests, GitHub Actions CI, typecheck-clean, Sentry-wired, structured logs
- Multi-tenant SaaS architecture with proper RLS, role-aware UI, plan-feature gating, seat licensing — most AI-tutor competitors haven't done this
- Provider-agnostic LLM adapter — instructors are not vendor-locked
- The rapid Phase A→I→Phase 2 pace shows a tight build cadence
- Detailed ROADMAP with explicit "BLOCKED ON YOU" items shows operational maturity
- Brutalist design system signals craft — competitors don't look like this

---

---

## 21. Questions Another AI Should Ask Me

### Product

- Who is the canonical first paying customer? A solo instructor, a department, or an enterprise pilot?
- Is the AI Chatrail Generator currently the headline feature or a behind-the-scenes shortcut?
- Should certificates be cryptographically-signed (verifiable on-chain) or is the current verification-code URL sufficient?
- What's the policy on AI-content disclosure to learners — should learners always know which bot is which model?
- Is there appetite for "learner-AI-tutor" features beyond chat (audio, vision, voice, screen-share) within the next year?

### Business model

- Is the priority to grow free → instructor-paid conversion, or to land enterprise / institutional deals?
- What ACV are you optimistic about for `org_dept` and `org_school` plans in year 1?
- Should the platform take a fee on learner-paid programs? If so, what %?
- Is annual billing on the near roadmap?

- How do you feel about a marketplace for shared Chatrails — community-built, founder-curated, or off-limits?

## Users

- What's your target segment-mix in pilot phase: higher-ed, K-12, corporate L&D, healthcare training, solo creator?
- Are you optimizing for the creator experience or the learner experience first?
- Do you expect orgs to bring their own LLM keys, or to consume bundled tokens?

## Pricing

- The free tier is generous (50K tokens, 5 learners). Is that intentional for activation, or should it be tighter?
- Should there be a launch promotion for the first 6 months?
- How aggressive should you be with the "monthly token budget" caps — soft (warn) or hard (cut off)?

## Technical architecture

- Is staying on Netlify the long-term plan, or is Vercel / a Render / Fly.io move likely if Lambda timeouts bite?
- Is there appetite for a background job queue (Inngest / BullMQ / SQS / Trigger.dev), or stay synchronous longer?
- How important is data residency for early customers (US-only, EU-required)?
- Should the AI generator be stream-output or stay non-streaming?

## AI behavior

- What's the policy on prompt-injection-resistance? Adversarial test bots?
- Should AI grading be opt-in per-bot (already configurable) or per-org by policy?
- Is BYO-LLM-key a near-term priority or a deferred enterprise feature?

## Compliance

- Is FERPA a near-term blocker for higher-ed sales, or does the segment tolerate informal compliance for pilots?
- Does anyone want to use Chatrail with PHI? If so, BAA work is on the critical path.
- Is there a target SOC 2 timeline?
- Are EU customers in the first-12-months pipeline? (drives DPA priority)

### Marketing

- Are you doing founder-led sales for the first 10 customers or building a marketing engine first?
- What channels: LinkedIn, X / Twitter, Substack, podcast tour, conference circuit, paid?
- Will there be a narrative essay / manifesto explaining the chatbot-native LMS thesis?

### Roadmap

- Of the Phase 5 enterprise features (LTI / SSO / SCORM / SOC 2), which one unblocks the biggest deal you have in the pipeline?
- Is mobile-app a 6-month priority or 18-month?
- Is internationalization (i18n) on the radar in year 1?

---

---

## 22. Best Prompt to Use With This Context Packet

Paste this into any AI chat after uploading the document:

> *I have uploaded a project context packet for this software project (Chatrail — a chatbot-native LMS). Read it carefully and treat it as the source of truth for our discussion. Help me think through the project using that context. If something is missing, ask targeted questions. Do not assume facts that are not in the packet. When giving recommendations, separate product advice, business advice, technical advice, and risks. When you reference files or routes, use the paths from the packet. When you propose changes to features, name the relevant table, route, or component from the packet. Begin by giving me a one-paragraph summary of what you learned from the packet, then ask me what I want to focus on.*

### Variant prompts for specific work types

- **Coding session:** "I have uploaded a project context packet for Chatrail. Read it as the source of truth for our codebase. We are going to work on [feature/file/bug]. Confirm you understand the relevant architecture before proposing code. Cite the specific table, route, or component from the packet you're touching."
- **Marketing brainstorm:** "I have uploaded a project context packet for Chatrail. Use sections 4 (Vision), 5 (Business model), 6 (Market), and 20 (Marketing). Help me draft [LinkedIn post / cold email / homepage hero / pricing page hero / segment-specific landing]. Stay grounded in what the product actually does."
- **Pricing review:** "I have uploaded a project context packet for Chatrail. Use sections 5 (Business model) and 6 (Market). Critique the current pricing structure, propose a better one if needed, and explain trade-offs. Don't invent features that aren't in the packet."

- **Compliance / legal:** "I have uploaded a project context packet for Chatrail. Use sections 14, 15, 16. Identify what's missing for [FERPA / GDPR / HIPAA / SOC 2] readiness. List blockers in priority order with rough effort estimates."
- **Pilot recruitment:** "I have uploaded a project context packet for Chatrail. Help me draft an outreach sequence to [segment] for a pilot. Stay grounded in the actual feature set. Be honest about what's beta vs production-ready."

---

---

## 23. Open Questions

The following could not be determined from the repo alone and likely need founder input:

1. Is the `chatquest.app` custom domain registered? If so, when does DNS get pointed at Netlify?
2. What is the **support email** — `security@chatquest.app` is in `SECURITY.md` but its operational status is Unknown.
3. Is **Stripe live mode** wired (products + price IDs created) or still pending?
4. Are there any **existing pilot customers** today, paid or comp'd?
5. What is the **target ACV** by plan tier — is `org_school` realistic at $1,999, or aspirational?
6. Is there a **marketing site CMS** plan, or do all content updates ship via code?
7. What's the **brand decision** on the legacy `chatquest` package name vs the product `Chatrail`?
8. Is the **AUP / Terms / Privacy** text counsel-reviewed, or scaffold awaiting review?
9. Is **PostHog / Plausible** the planned analytics tool, or are they undecided?
10. Are **DPAs** in place with Anthropic / OpenAI / Google, or still to-do?
11. Who is the **first canonical paying customer profile** — solo creator, dept, or enterprise?
12. What's the **revenue / ARR target** for end of year 1?
13. What's the **fee % on learner-paid programs**, if Chatrail charges one?
14. Is there a **founding team** beyond the solo creator, or is this solo today?
15. What's the **fundraising posture** — bootstrapped, seed-stage, or pre-seed?
16. Is the **brutalist design** non-negotiable, or open to a "softer institutional" alt theme for enterprise buyers?
17. Are there **referral / ambassador programs** planned for instructional designers?
18. What is the **content moderation** stance for learner-uploaded media or learner messages?
19. What's the **data retention policy** for conversations after a course ends?
20. Is **multilingual support** in year 1 or year 2?

## 24. Source Notes

This packet was generated by inspecting the following files in the repo:

- `README.md` — primary source for stack, setup, MVP scope
- `ROADMAP.md` — primary source for roadmap, BLOCKED ON YOU items, phasing, security gaps
- `CHANGELOG.md` — Phase A–I shipped features
- `SECURITY.md` — security policy + scope + reporting
- `package.json` — dependency list, scripts, Node version
- `.env.example` — environment variable names (values intentionally not read)
- `supabase/migrations/00010000000001_extensions.sql` through `00010000000040_phase_t_notifications.sql` — full DB schema, RLS policies, plan rows, all 13 migrations
- `app/(marketing)/page.tsx` — landing-page features and copy
- `app/(marketing)/pricing/page.tsx` — pricing reads from `plans` table via PostgREST
- Route tree under `app/(app)/` and `app/api/` — to enumerate pages and endpoints
- `lib/llm/provider.ts`, `lib/llm/cost.ts`, `lib/llm/usage.ts`, `lib/llm/generate-chatrail.ts` — LLM abstraction
- `lib/path/progress.ts` — pure path-progress engine
- `lib/billing/gate.ts`, `lib/usage/check.ts` — plan-feature + usage gates
- `components/brutalist/*` — design system primitives

Plus the running session context from this codebase's recent development (which has built and shipped 30+ commits this week).

**Secrets excluded.** Environment variable names mentioned, values never read or copied. `.env.local` was intentionally not opened.

---

*End of Project Context Packet · Chatrail · Generated 2026-05-05*